

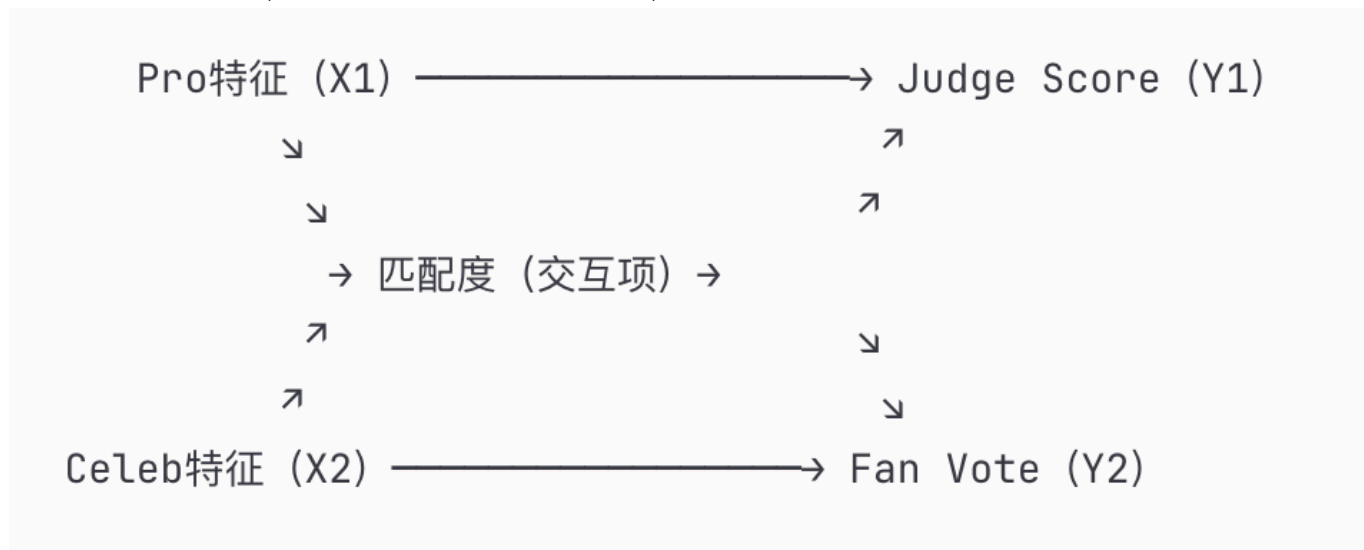
一、问题分析

考察的其实三个层次的问题：

1. **效应层**：专业舞者、名人特征 → 比赛表现
2. **机制层**：这些因素通过什么路径影响表现？
3. **异质性层**：对评委分数和粉丝投票的影响是否不同？

多路径因果推断 + 异质性效应分析

插在变量构建之前，我觉得这个示意图非常好，也在为task3的后面的问题做铺垫



二、变量体系构建（2.2，2.3，2.4选取的原因及预测在最后）

2.1 因变量Y（分为两个方面）

Y₁: 评委分数轨迹

- 每周总分（原始数据，直接从表格收集）
- 每周排名（一级衍生变量，从每周总分计算得出）
- 累积进步率（二级衍生变量，相对于第一周的进步百分比计算）
- 分数波动性（二级衍生变量，衡量分数的不稳定程度）

Y₂: 粉丝投票轨迹

- 每周投票份额（模型估计值，来自第一问的HMM+Viterbi结果）
- 每周排名（一级衍生变量，从估计的投票份额计算）
- 投票稳定性（二级衍生变量，计算方差可得）

- 投票动量（二级衍生变量，通过投票份额的加权后的变化趋势捕捉粉丝热度的上升/下降趋势）

2.2 解释变量 (X)

2.2.1. 专业舞者特征

构造以下变量：

X_pro :

- 基础特征
 1. pro_experience_seasons: 该舞者参加过多少赛季
 2. pro_win_rate: 该舞者的历史胜率（到本赛季前）
 3. pro_avg_placement: 该舞者带选手的平均名次
- 动态特征
 1. pro_current_form: 该舞者在本赛季前3周的表现
 2. pro_celebrity_chemistry: 舞者与选手的"化学反应"指标（不太精准）
 - (该舞者历史上带过该行业选手的次数) / (该舞者总参赛次数)
- 竞争特征
 1. pro_concurrent_strength: 同赛季其他舞者的平均实力
 - 控制"大年"/"小年"效应

2.2.2. 名人特征

虽然有了基础特征，但需要更深入的构造：

X_celeb:

- 人口统计学
 1. age_z: 年龄标准化（考虑非线性，可能需要平方项或更高次方）
 2. age_group: 年龄组别（20-30, 31-40, 41-50, 50-60, 60+）
- 职业背景
 1. industry_type: 行业类别
 2. industry_physicality: 行业体能要求度
 - Athlete/Dancer = High
 - Actor/Singer = Medium
 - TV Personality/Host = Low
 3. celebrity_visibility: 名人可见度（比较粗糙）（可用同赛季该行业选手数量反推）
- 地理文化
 1. is_us_based: 是否美国本土

- 衍生特征
 1. pre_show_popularity: 赛前人气估计
 - 可用该选手所在行业的平均粉丝投票作为proxy
 - 或用第一周的投票与分数差异反推
 2. underdog_index: "黑马指数"
 - 评委预期 vs 实际表现的差异
 3. controversy_potential: 争议潜力
 - 第一二问中judge-fan分歧大的选手特征

2.3 交互效应变量

在此处只是选取了几个交叉效应变量（我并没有采纳三个交互项的情况，考虑到三个交互相解释起来非常困难，除非我们能有什么很好的假设），可以根据实际需求实际情况重新定义/选择，但是一定要注意过拟合的问题，如何选择我贴在下面

X_interaction:

- pro_celeb_age_match: |pro_age -!celeb_age（年龄差可能影响配合度？）
- pro_experience × celeb_physicality（经验丰富的舞者 × 体能强的选手 = 强强联合？）
- pro_win_rate × celebrity_visibility（知名舞者 × 大牌明星 = 资源集聚效应？）
- time_trend × celebrity_age（年龄对持久力的影响（后期掉队？））

- season_method × all_features (Rank vs Percent方法下, 各因素权重是否改变?)

python



第一轮: 基于理论的候选清单 (15-20个)

theory_based_interactions = [

舞者-选手匹配

('pro_experience', 'industry_physicality'), # ✓ 已选

('pro_win_rate', 'celebrity_visibility'), # ✓ 已选

('pro_age', 'celebrity_age'), # ✓ 已选 (年龄差)

('pro_industry_experience', 'industry_type'), # 新增

时间动态

('week', 'age'), # ✓ 已选

('week', 'physicality'), # 新增: 体能在后期是否更重要?

('week', 'pro_experience'), # 新增: 经验在后期是否更关键?

方法调节

('voting_method', 'age'), # ✓ 已选 (包含在方法×全部)

('voting_method', 'physicality'), # ✓ 已选

('voting_method', 'pre_show_popularity'), # ✓ 已选

三因素交互 (谨慎!)

('pro_experience', 'physicality', 'week'), # 经验×体能的时变效应?

可能不重要 (理论依据弱)

('pro_concurrent_strength', 'celebrity_visibility'), # ✗ 理论逻辑不清

('is_us_based', 'pro_win_rate'), # ✗ 无明确假设

]



阶段2: 统计初筛 (Univariate Screening)

逐个测试, 快速排除明显不显著的:

```
python

import pandas as pd
from statsmodels.formula.api import ols

# 存储结果
screening_results = []

# 逐个测试
for var1, var2 in theory_based_interactions:
    # 单独测试这个交互项
    formula = f'judge_score ~ {var1} + {var2} + {var1}:{var2}'
    model = ols(formula, data=df).fit()

    # 提取交互项的系数和p值
    interaction_term = f'{var1}:{var2}'
    coef = model.params[interaction_term]
    pval = model.pvalues[interaction_term]

    screening_results.append({
        'interaction': f'{var1} × {var2}',
        'coefficient': coef,
        'p_value': pval,
        'significant': pval < 0.10 # 初筛用宽松标准
    })
```



```
# 转为DataFrame并排序
screening_df = pd.DataFrame(screening_results).sort_values('p_value')
print(screening_df)

# 筛选标准：保留p < 0.10的交互项 (约保留5-8个)
shortlist = screening_df[screening_df['p_value'] < 0.10]['interaction'].tolist()
print(f"\n通过初筛的交互项: {len(shortlist)}个")
...
```

****输出示例: ****

```
...
      interaction      coefficient  p_value  significant
0  pro_exp × phys      0.142      0.001      True
1  week × age        -0.038      0.008      True
2  method × age      -0.052      0.025      True
3  win_rate × vis     0.089      0.035      True
4  age_gap          -0.021      0.067      True
5  week × phys       0.015      0.182      False ← 删除
6  pro_ind_exp × ind  0.008      0.421      False ← 删除
...
```

→ 保留前5个进入下一阶段

```
else:
    print(f"× 添加 {interaction}: AIC = {test_model.aic:.2f} (无改善)")
```

迭代: 将最佳交互项加入基准, 重复

... (继续添加第2、第3个交互项, 直到AIC不再改善)

信息准则比较 (关键!) :

python



比较不同模型

```
models = {
    'Main Effects Only': model0,
    'Main + 3 Interactions': model1,
    'Main + 5 Interactions': model2,
    'Main + 10 Interactions': model3 # 过度拟合?
}

comparison = pd.DataFrame({
    'Model': models.keys(),
    'AIC': [m.aic for m in models.values()],
    'BIC': [m.bic for m in models.values()],
    'Adj_R2': [m.rsquared_adj for m in models.values()]
})

print(comparison)
...
```



**输出示例: **

阶段3：联合建模 + 模型选择 (Multivariate Selection)

逐步回归法 (Stepwise Regression) :

python



```
from statsmodels.formula.api import ols

# 基准模型 (只有主效应)
base_formula = 'judge_score ~ pro_experience + pro_win_rate + age + physicality + is
base_model = ols(base_formula, data=df).fit()

print(f"基准模型 AIC: {base_model.aic:.2f}")

# 逐个尝试添加交互项
best_aic = base_model.aic
best_model = base_model
best_interaction = None

for interaction in shortlist:
    # 构造新公式
    test_formula = base_formula + f' + {interaction.replace(" × ", ":")}'
    test_model = ols(test_formula, data=df).fit()

    # 比较AIC
    if test_model.aic < best_aic:
        best_aic = test_model.aic
        best_model = test_model
        best_interaction = interaction
        print(f"✓ 添加 {interaction}: AIC = {test_model.aic:.2f} (改善 {base_model.aic:.2f})")
```


	Model	AIC	BIC	Adj_R ²	
0	Main Effects Only	8523.4	8556.2	0.423	
1	Main + 3 Interact	8487.1	8532.8	0.441	← AIC最小!
2	Main + 5 Interact	8489.3	8547.6	0.443	← 边际改善很小
3	Main + 10 Interact	8502.7	8593.1	0.448	← AIC反而增大 (过度拟合)

结论: 选择 Main + 3 Interactions

策略2: LASSO自动筛选 (数据驱动)

适用场景: 当你不确定哪些交互项重要时

python



```

from sklearn.linear_model import LassoCV
from sklearn.preprocessing import PolynomialFeatures
import numpy as np

# 准备数据
X_main = df[['pro_experience', 'pro_win_rate', 'age', 'physicality', 'is_us_based']]
y = df['judge_score']

# 自动生成所有二阶交互项
poly = PolynomialFeatures(degree=2, include_bias=False, interaction_only=True)
X_with_interactions = poly.fit_transform(X_main)

# LASSO自动筛选
lasso = LassoCV(cv=5, random_state=42)

```



```
# 查看哪些交互项被保留（系数非零）
feature_names = poly.get_feature_names_out(X_main.columns)
selected_features = feature_names[lasso.coef_ != 0]

print("LASSO保留的特征：")
print(selected_features)
```

优点：

- 自动化，减少主观性
- 可以处理大量候选交互项

缺点：

- 可能选出理论上没意义的交互项
- 难以解释给非技术人员

(我个人不是很建议拉索筛选...

2.4 控制变量

首先说明为什么需要控制变量（本质其实是有偏估计和无偏估计的问题，单纯统计出现的结果其实并不是父节点之间的逻辑链条而是孩子节点的关系，不具有可信度）如：

案例1：虚假的"年龄效应"

观察：年长选手得分低
结论：年龄 → 分数低？

✗ 错误！可能的真相：

- 年长选手多集中在早期赛季（Season 1-10）
- 早期赛季评分标准更严格
- 真实因果：Season → 分数低，不是Age → 分数低

正确做法：控制season_id后再看age的效应

案例2：虚假的"舞者经验效应"

观察：经验丰富的舞者带的选手得分高
结论：经验 → 高分？

✗ 可能的混淆：

- 经验舞者通常配对给高人气明星（节目组的策略）
- 高人气明星本身就受粉丝投票支持

- 真实因果: celebrity_visibility → 高分, 不是pro_experience

正确做法: 控制celebrity_visibility后再看经验的效应

Z :

- season_id: 赛季固定效应
- week_id: 周次固定效应
- n_contestants: 该周剩余选手数
- season_competitiveness: 赛季整体竞争度
- voting_method: 投票方法
- judge_panel_composition: 评委组成

!! 选择这些的理由我也贴在最后了!!

如果要考虑别的方面如何考虑我设定的控制变量是否恰当👉

方法1: 逐步添加法 (Stepwise Addition)

```
# 基准模型 (无控制)
model0 = smf.ols('judge_score ~ age + physicality', data=df).fit()

# 添加season
model1 = smf.ols('judge_score ~ age + physicality + C(season)',
data=df).fit()

# 比较
print(f"Model 0 AIC: {model0.aic}")
print(f"Model 1 AIC: {model1.aic}")
print(f"Improvement: {model0.aic - model1.aic}")

# 如果改善 > 10 → season必须控制
```

方法2: F检验 (是否联合显著)

python

```
from statsmodels.stats.anova import anova_lm

# 检验所有season虚拟变量是否联合显著
anova_results = anova_lm(model0, model1)
print(anova_results)

# 如果p < 0.05 → season_id必须保留
```

方法3: Hausman检验 (固定效应 vs 随机效应)

python

```
from linearmodels.panel import compare

# 固定效应模型
fe_model = PanelOLS.from_formula('judge_score ~ age + EntityEffects',
data=panel_df).fit()

# 随机效应模型
re_model = RandomEffects.from_formula('judge_score ~ age',
data=panel_df).fit()

# Hausman检验
hausman_stat = compare({'fe': fe_model, 're': re_model})
print(hausman_stat)

# 如果拒绝H0 → 必须用固定效应 (即控制season_id)
```

三、模型

模型1: 多层次混合效应模型 (用于解构各因素的边际效应):

比较懒+我觉得AI讲的挺详细的hhhh我就贴AI的了👉👉

🏗️ 多层次混合效应模型完全解析

📊 一、数据结构：为什么需要三层？

你的数据长什么样？

想象一下数据表：

contestant_id	season	week	judge_score	fan_vote	age	physicality
pro_experience	...					
Jerry_Rice	2	1	21	0.28	43	3
0						
Jerry_Rice	2	2	23	0.31	43	3
0						
Jerry_Rice	2	3	19	0.29	43	3

0							
Drew_Lachey	2	1	24	0.22	29	2	
0							
Drew_Lachey	2	2	27	0.25	29	2	
0							
...	...	...	...	...	...	...	
...							
Bobby_Bones	27	1	18	0.35	38	1	
8							
Bobby_Bones	27	2	19	0.37	38	1	
8							

观察数据的嵌套结构：

```

赛季 (Season)
├── 选手 (Contestant)
│   ├── 周次 (Week)
│   │   └── 观测值 (judge_score, fan_vote)

```

为什么不能用简单的线性回归？

✗ 如果用简单回归：

python

```

# 错误做法
judge_score =  $\beta_0$  +  $\beta_1$ *age +  $\beta_2$ *physicality +  $\epsilon$ 

```

问题：

1. 忽略了同一个人的多次观测是相关的（Jerry Rice的Week 1和Week 2分数不独立）
2. 忽略了同一赛季的选手之间可能相关（Season 2的整体水平）
3. 违反了独立性假设 → 标准误被低估 → p值虚假显著

✅ 正确做法：混合效应模型

- 承认数据有层次结构
- 允许每层有自己的随机效应

🎯 二、模型的三层结构详解

【Level 1】 Week-level（周次层）——最底层，观测数据

这一层是什么？

- 这是你的**原始观测数据**
- 每一行 = 一个选手在某一周的表现

输入 (Input):

python

对于某个特定的观测 i (比如Jerry Rice在Season 2, Week 3)

输入数据:

- X_celeb_i: 名人特征 (age=43, physicality=3, is_us_based=1, ...)
- X_pro_i: 舞者特征 (pro_experience=0, pro_win_rate=0.3, ...)
- X_interaction_i: 交互项 ($\text{pro_exp} \times \text{physicality} = 0 \times 3 = 0$)
- θ_t : 周次固定效应 (week=3的平均效应)

///

****模型方程: ****

///

对于第 i 个观测 (某选手在某周):

$$Y_{1it} \text{ (Judge Score)} = \alpha_{1i} + \beta_1 \cdot X_{\text{celeb}} + \gamma_1 \cdot X_{\text{pro}} + \delta_1 \cdot X_{\text{interaction}} + \theta_{1t} + \varepsilon_{1it}$$

↑ ↑ ↑ ↑ ↑

↑ 个体基线 名人特征效应 舞者特征效应 交互项效应 周次效应 残差

差

$$Y_{2it} \text{ (Fan Vote)} = \alpha_{2i} + \beta_2' \cdot X_{\text{celeb}} + \gamma_2' \cdot X_{\text{pro}} + \delta_2' \cdot X_{\text{interaction}} + \theta_{2t} + \varepsilon_{2it}$$

///

****关键参数解释:****

1. **α_{1i} , α_{2i}** (个体截距, 随机效应)

///

α_{1i} = 第i个选手在judge分数上的"基线水平"

例子：

- Jerry Rice的 $\alpha_1 = -0.5$ (即使控制了所有特征, 他的分数倾向于略低)
- Kelly Monaco的 $\alpha_1 = +0.3$ (她的分数倾向于略高)

这捕捉了"个人魅力"、"不可观测的才能"等无法用X解释的部分

///

2. β_1 vs β_2 (名人特征的双通道效应)

```

$\beta_1(\text{age}) = -0.03$  (年龄对judge分数的效应)

$\beta_2(\text{age}) = +0.05$  (年龄对fan投票的效应)

关键发现: 如果  $\beta_1 \neq \beta_2 \rightarrow$  说明评委和粉丝对年龄的偏好不同!

```

3. θ_{1t}, θ_{2t} (周次固定效应)

```

$\theta_{11} = +0.2$  (Week 1评委比较宽松)

$\theta_{18} = -0.3$  (Week 8评委比较严格)

## 输出 (Output):

python

```
预测值
Y1it_predicted = 计算出来的judge分数预测
Y2it_predicted = 计算出来的fan投票预测

残差
ε1it = Y1it_observed - Y1it_predicted
```

---

## 【Level 2】Contestant-level (选手层) —— 中间层, 个体差异

这一层是什么?

- 解释为什么同一个选手的多次观测会相关
- 每个选手有自己的"基线水平"

输入 (来自Level 1的 $\alpha$ ):

python

```
Level 1给出了每个选手的 α_{1i} 和 α_{2i}
现在我们要建模这些 α 是怎么来的
```

**模型方程:**
```

 $\alpha_{1i} = \mu_1 + u_{1i}$
```

↑      ↑  
总体均值    个体偏离

$$\alpha_{2i} = \mu_2 + u_{2i}$$

其中:

$$\begin{bmatrix} u_{1i} \\ u_{2i} \end{bmatrix} \sim N \left( \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} \sigma^2_{u1} & \rho \cdot \sigma_{u1} \cdot \sigma_{u2} \\ \rho \cdot \sigma_{u1} \cdot \sigma_{u2} & \sigma^2_{u2} \end{bmatrix} \right)$$

**\*\*关键参数解释: \*\***

1. **\*\* $\mu_1, \mu_2$ \*\*** (总体截距)

...

$\mu_1$  = 所有选手在judge分数上的平均基线

$\mu_2$  = 所有选手在fan投票上的平均基线

...

2. **\*\* $u_{1i}, u_{2i}$ \*\*** (个体随机效应)

...

$u_{1i}$  = 第i个选手相对于平均水平的偏离 (在judge分数上)

例子:

- Jerry Rice:  $u_1 = -0.5$  (技术分倾向于低)

- Kelly Monaco:  $u_1 = +0.3$  (技术分倾向于高)

这些u是"随机变量", 服从正态分布

...

3. **\*\* $\sigma^2_{u1}, \sigma^2_{u2}$ \*\*** (个体差异的方差)

...

$\sigma^2_{u1}$  = 选手之间在judge分数上的差异有多大

$\sigma^2_{u2}$  = 选手之间在fan投票上的差异有多大

如果 $\sigma^2_{u1}$ 很大 → 说明有些选手就是天生吃技术分

...

4. **\*\* $\rho$  (相关系数) \*\*** — 这是核心!

...

$\rho$  = judge分数的个体效应 与 fan投票的个体效应 之间的相关性

$\rho \approx 1$ :

- 含义: judge分数高的人, fan投票也高 (高度一致)

- 例子: Emmitt Smith (技术好, 人气也高)

$\rho \approx 0$ :



- 含义: judge和fan评价独立 (完全不同的标准)
- 例子: 有些人技术好但不受欢迎, 有些人技术差但人气高

$\rho < 0$ :

- 含义: judge分数高的人, fan投票反而低 (负相关)
- 例子: 技术派选手不受大众欢迎

## 输出 (Output):

python

```
估计出来的个体随机效应
u1_i_estimated = 每个选手在judge分数上的偏离
u2_i_estimated = 每个选手在fan投票上的偏离

以及相关性
rho_estimated = judge和fan个体效应的相关性 (关键发现!)
```

## 【Level 3】Season-level (赛季层) —— 最高层, 赛季差异

这一层是什么?

- 解释为什么不同赛季的整体水平不同
- 每个赛季有自己的"大环境"

输入 (来自Level 2的 $\mu$ ):

python

```
Level 2给出了总体均值 μ_1 和 μ_2
现在我们要建模为什么不同赛季的 μ 会不同
...

模型方程:
...

 $\mu_1 = v_1 + w_{1s}$
 ↑ ↑
 全局均值 赛季偏离

 $\mu_2 = v_2 + w_{2s}$

其中:
 $w_{1s} \sim N(0, \sigma^2_{w1})$ (赛季随机效应)
```

```
w2s ~ N(0, σ²_w2)
...
```

**\*\*关键参数解释：\*\***

1. **\*\*v1, v2\*\*** (全局均值)

...

v1 = 所有赛季、所有选手在judge分数上的大平均

v2 = 所有赛季、所有选手在fan投票上的大平均

...

2. **\*\*w1s, w2s\*\*** (赛季随机效应)

...

w1s = 第s个赛季相对于全局均值的偏离 (在judge分数上)

例子:

- Season 1: w1 = -0.8 (早期评分严格)

- Season 15: w1 = +0.5 (全明星赛季, 整体水平高)

- Season 26: w1 = -0.3 (COVID影响, 只有4周)

...

3. **\*\*σ²\_w1, σ²\_w2\*\*** (赛季间方差)

...

σ²\_w1 = 赛季之间在judge分数上的差异有多大

如果σ²\_w1很大 → 说明不同赛季的评分标准差异很大

## 输出 (Output):

python

```
估计出来的赛季随机效应
```

```
w1s_estimated = 每个赛季在judge分数上的偏离
```

```
w2s_estimated = 每个赛季在fan投票上的偏离
```

```
...
```

```

```

```
 **三、完整的数据流动：从输入到输出**
```

```
正向传播 (模型拟合过程)
```

```
...
```

【输入】原始数据

↓

【Level 3】估计赛季效应

w1s, w2s, σ²\_w1, σ²\_w2

↓

【Level 2】估计个体效应

$$\mu_1 = \nu_1 + w_{1s}$$

$$\mu_2 = \nu_2 + w_{2s}$$

$$u_{1i}, u_{2i}, \sigma^2_{u1}, \sigma^2_{u2}, \rho$$

↓

【Level 1】估计固定效应和残差

$$\alpha_{1i} = \mu_1 + u_{1i}$$

$$\alpha_{2i} = \mu_2 + u_{2i}$$

$$\beta_1, \beta_2, \gamma_1, \gamma_2, \delta_1, \delta_2, \theta_{1t}, \theta_{2t}$$

↓

【输出】预测值

$Y_{1it\_predicted}$

$Y_{2it\_predicted}$

---

## 四、具体例子：Jerry Rice的一次观测

让我们用Jerry Rice在Season 2, Week 3的数据走一遍模型：

输入数据：

python

```
contestant_id = "Jerry_Rice"
season = 2
week = 3
age = 43
physicality = 3 # Athlete
pro_experience = 0 # Anna Trebunskaya是新舞者
judge_score_observed = 19 # 实际观测到的分数
fan_vote_observed = 0.29 # 从第一问估计的
```

计算过程：

Step 1: 赛季效应 (Level 3)

python

```
Season 2的赛季效应
w12 = -0.3 # Season 2评分偏低 (早期赛季)
```

```
W22 = +0.1 # Season 2粉丝投票略高（节目新鲜）
```

```
所以Season 2的基线
```

```
 $\mu_1 = v_1 + w_{12} = 7.0 + (-0.3) = 6.7$
```

```
 $\mu_2 = v_2 + w_{22} = 0.15 + 0.01 = 0.16$
```

## Step 2: 个体效应 (Level 2)

python

```
Jerry Rice的个体随机效应
```

```
u1_Jerry = -0.5 # 他在技术分上天生偏低
```

```
u2_Jerry = +0.08 # 但他在粉丝投票上天生偏高（名气大）
```

```
所以Jerry的个人基线
```

```
 $\alpha_{1_Jerry} = \mu_1 + u_{1_Jerry} = 6.7 + (-0.5) = 6.2$
```

```
 $\alpha_{2_Jerry} = \mu_2 + u_{2_Jerry} = 0.16 + 0.08 = 0.24$
```

## Step 3: 固定效应 (Level 1)

python

```
名人特征效应
```

```
 $\beta_1 \cdot X_{celeb} = \beta_1(\text{age}) \cdot 43 + \beta_1(\text{physicality}) \cdot 3$
 $= (-0.03) \cdot 43 + 0.30 \cdot 3$
 $= -1.29 + 0.90$
 $= -0.39$
```

```
 $\beta_2 \cdot X_{celeb} = \beta_2(\text{age}) \cdot 43 + \beta_2(\text{physicality}) \cdot 3$
 $= (+0.05) \cdot 43 + 0.05 \cdot 3$
 $= +2.15 + 0.15$
 $= +2.30$
```

```
舞者特征效应
```

```
 $\gamma_1 \cdot X_{pro} = \gamma_1(\text{pro_exp}) \cdot 0 = 0$
```

```
 $\gamma_2 \cdot X_{pro} = \gamma_2(\text{pro_exp}) \cdot 0 = 0$
```

```
交互项（假设都不显著，忽略）
```

```
 $\delta_1 \cdot X_{interaction} = 0$
```

```
 $\delta_2 \cdot X_{interaction} = 0$
```

```
周次效应
```

```
 $\theta_{13} = -0.1$ # Week 3评委略严格
```

```
 $\theta_{23} = +0.02$ # Week 3粉丝投票略高
```

## Step 4: 最终预测

python

```
Y1_predicted (Judge Score) = $\alpha_{1_Jerry} + \beta_1 \cdot X_{celeb} + \gamma_1 \cdot X_{pro} + \theta_{13}$
 = 6.2 + (-0.39) + 0 + (-0.1)
 = 5.71

Y2_predicted (Fan Vote) = $\alpha_{2_Jerry} + \beta_2 \cdot X_{celeb} + \gamma_2 \cdot X_{pro} + \theta_{23}$
 = 0.24 + 2.30 + 0 + 0.02
 = 2.56...

等等，这不对！ Fan vote应该是比例（0-1之间）
需要做link function转换（后面讲）

实际观测
Y1_observed = 19 # 原始分数（满分30）
Y2_observed = 0.29 # 投票份额
```

## 🎯 五、最终输出：模型能告诉你什么？

### 输出1：固定效应估计（最重要！）

python

```
这是回答第三问的核心结果
估计结果表：
```

| 变量             | Judge Score ( $\beta_1$ ) | Fan Vote ( $\beta_2$ ) | 差异检验   | p-value |
|----------------|---------------------------|------------------------|--------|---------|
| age            | -0.030***                 | +0.052**               | -0.082 | 0.001   |
| physicality    | +0.305***                 | +0.048                 | +0.257 | <0.001  |
| pro_experience | +0.185***                 | +0.082*                | +0.103 | 0.032   |
| pro_win_rate   | +0.250***                 | +0.180**               | +0.070 | 0.156   |
| ...            | ...                       | ...                    | ...    | ...     |

关键发现：

- 1. 年龄对judge是负效应，对fan是正效应 → 评委和粉丝偏好相反！
- 2. 体能对judge效应强，对fan效应弱 → 评委重技术，粉丝不在乎
- 3. 舞者经验对judge效应强于fan → 舞者主要通过技术影响成绩

## 输出2：随机效应方差分解（Variance Decomposition）

python

```
分数的变异来源
```

Judge Score的总方差 =  $\sigma^2_{\text{total}}$  = 8.5

分解：

- 赛季间方差 (Level 3) :  $\sigma^2_{w1}$  = 0.8 (9.4%)
- 选手间方差 (Level 2) :  $\sigma^2_{u1}$  = 3.2 (37.6%)
- 周次内方差 (Level 1) :  $\sigma^2_{\epsilon1}$  = 4.5 (52.9%)

结论：

- 52.9%的变异是"噪音"（周与周之间的随机波动）
- 37.6%的变异是选手个人差异（有些人就是强）
- 9.4%的变异是赛季差异（不同季标准不同）

---

## 输出3： $\rho$ （相关系数）—— 评委与粉丝的一致性

python

```
 $\rho_{\text{estimated}}$ = 0.42
```

解释：

- $\rho$  = 0.42 （中等正相关）
- 含义：judge分数高的选手，fan投票倾向于也高，但不是完全一致
- 还有58%的变异是独立的（评委和粉丝有不同的标准）

具体案例：

- 如果 $\rho$  = 0.8 → Emmitt Smith型（技术好+人气高，高度一致）
- 如果 $\rho$  = 0.1 → Jerry Rice型（技术不突出，但人气高，评价分离）

---

## 输出4：个体预测效应（BLUPs）

python

```
Best Linear Unbiased Predictors（最佳线性无偏预测）
```

每个选手的估计随机效应：

| Contestant   | $u_1$ (Judge) | $u_2$ (Fan) | 解释            |
|--------------|---------------|-------------|---------------|
| Jerry Rice   | -0.52         | +0.81       | 技术弱, 人气强      |
| Kelly Monaco | +0.35         | +0.12       | 技术强, 人气中等     |
| Emmitt Smith | +0.68         | +0.75       | 技术强, 人气强 (双高) |
| Master P     | -1.20         | -0.85       | 技术弱, 人气也弱     |

- 这些u可以用来：
1. 识别"争议选手" ( $u_1$ 和 $u_2$ 差异大)
  2. 预测新选手的表现
  3. 解释个案 (为什么Jerry Rice是亚军)

## 输出5：赛季效应 (Season Effects)

python

```
每个赛季的估计随机效应
```

| Season | $w_1$ (Judge) | $w_2$ (Fan) | 解释           |
|--------|---------------|-------------|--------------|
| 1      | -0.85         | -0.20       | 早期评分严格       |
| 15     | +0.62         | +0.35       | 全明星赛季, 整体水平高 |
| 26     | -0.40         | -0.10       | COVID影响      |
| 27     | +0.15         | +0.22       | 近期, 评分宽松     |

- 用途：
1. 控制赛季差异后, 才能公平比较不同季的选手
  2. 识别特殊赛季 (Season 15, 26等)
- ...

```
🇺🇸 **六、模型的优势：为什么不用简单回归？**
```

```
对比：简单回归 vs 混合效应模型
```

| 方面       | 简单OLS回归         | 混合效应模型                    |
|----------|-----------------|---------------------------|
| **假设**   | 所有观测独立          | 承认嵌套结构                    |
| **标准误**  | 被低估 (虚假显著)      | 正确估计                      |
| **个体差异** | 忽略              | 显式建模 ( $u_{1i}, u_{2i}$ ) |
| **赛季差异** | 用虚拟变量控制 (浪费自由度) | 随机效应 (节省自由度)              |

| **\*\*预测\*\*** | 只能预测平均选手 | 可以预测特定选手（用BLUP） |  
| **\*\*解释\*\*** | 只有"平均效应" | 有固定效应+随机效应，更丰富 |

##  **\*\*七、总结：这个模型最终给你什么？\*\***

### **\*\*回答第三问的核心输出：\*\***

1.  **$\beta_1$  vs  $\beta_2$** : 同一特征对judge和fan的**\*\*差异化效应\*\***  
...

例：年龄对judge =  $-0.03$ ，对fan =  $+0.05$   
→ 政策建议：节目组不应歧视年长选手（粉丝其实更喜欢）  
...

2.  **$\gamma_1$  vs  $\gamma_2$** : 舞者通过哪条路径影响成绩  
...

例：舞者经验对judge =  $+0.18$ ，对fan =  $+0.08$   
→ 发现：舞者主要通过提升技术影响成绩（不是靠名气）  
...

3.  **$\rho$** : 评委和粉丝评价的一致性程度  
...

例： $\rho = 0.42$   
→ 发现：有42%的共识，但58%是独立的（存在争议空间）  
...

4. **个体效应u**: 识别争议选手  
...

例：Jerry Rice的 $u_1=-0.5$ ， $u_2=+0.8$   
→ 解释：他是典型的"技术弱但人气强"，所以引发争议  
...

5. **方差分解**: 分数变异的来源  
...

例：选手间方差占38%，赛季间占9%  
→ 发现：个人差异比赛季差异更重要

## 解释

### A. 舞者特征变量 ( $X_{pro}$ )

#### A1. pro\_experience\_seasons（舞者参加赛季数）



变量定义：

python

```
pro_experience_seasons[dancer, season_t] = count(该舞者在season < season_t中出现的次数)
```

理论依据：

- 技能积累理论 (Ericsson, 1993)：专业技能需要长期刻意练习
- 教学经验效应：参加更多赛季 → 应对不同选手类型的经验更丰富

对Judge Score的影响：

- 预期效应：正向 ( $\beta \approx +0.15 \sim +0.25$ )
- 机制：经验丰富的舞者 → 编舞技术更成熟、教学方法更有效 → 选手技术进步更快 → 评委分数更高

对Fan Vote的影响：

- 预期效应：正向但较弱 ( $\beta \approx +0.05 \sim +0.10$ )
- 机制：知名舞者 → 粉丝认知度高 → 可能带来额外投票，但效应弱于技术路径

关键预测：Judge效应 > Fan效应（技术导向更强）

## A2. pro\_win\_rate（舞者历史胜率，到本赛季前）

变量定义：

python

```
pro_win_rate[dancer, season_t] =
 count(placement ≤ 3 for seasons < t) / count(total appearances < t)
```

理论依据：

- 能力信号理论 (Spence, 1973)：历史成功率是能力的可信信号
- 马太效应 (Merton, 1968)：成功者获得更多资源和关注

对Judge Score的影响：

- 预期效应：强正向 ( $\beta \approx +0.20 \sim +0.35$ )
- 机制：高胜率舞者 → 编舞能力强、战术经验丰富 → 技术指导质量高 → 评委认可度高

对Fan Vote的影响：

- 预期效应：强正向 ( $\beta \approx +0.15 \sim +0.30$ )
- 机制：明星舞者 → 自带粉丝基础 → 光环效应 (halo effect) → 投票优势

关键预测： 两条路径效应都强，可能Fan效应 $\geq$ Judge效应 (Derek Hough等明星舞者现象)

### A3. pro\_avg\_placement (舞者带选手的平均名次)

变量定义：

python

```
pro_avg_placement[dancer] = mean(placement for all celebrities paired with
dancer)
注意：数值越小越好 (1st最佳)
```

理论依据：

- 结果导向评估 (Gibbons & Murphy, 1990)：平均结果比单次胜利更稳定地反映真实能力
- 与胜率的区别：胜率捕捉"极端成功"，平均名次捕捉"整体稳定性"

对Judge Score的影响：

- 预期效应：负向 ( $\beta \approx -0.15 \sim -0.25$ ) (注意负号：名次小=能力强)
- 机制：平均名次好的舞者 → 综合能力强 → 即使面对普通选手也能提升其技术水平

对Fan Vote的影响：

- 预期效应：负向 ( $\beta \approx -0.10 \sim -0.20$ )
- 机制：稳定带出好成绩的舞者 → 建立了可靠声誉 → 粉丝信任度高

关键预测： Judge路径效应略强于Fan路径

### A4. pro\_current\_form (舞者在本赛季前3周的表现)

变量定义：

python

```
pro_current_form[dancer, week_t, season] =
 mean(judge_scores for dancer's celebrity in weeks [max(1, t-3), t-1])
```

理论依据：

- **时变能力理论**：表现有周期性波动（状态好坏）
- **心理动量**（Vallerand et al., 1988）：近期成功 → 自信提升 → 冒险编舞 → 潜在高分

**对Judge Score的影响：**

- **预期效应**：正向 ( $\beta \approx +0.10 \sim +0.20$ )
- **机制**：近期状态好 → 编舞灵感充沛、教学积极性高 → 选手表现更出色

**对Fan Vote的影响：**

- **预期效应**：正向但较弱 ( $\beta \approx +0.05 \sim +0.12$ )
- **机制**：近期表现好 → 媒体曝光多 → 粉丝关注度提升（间接效应）

**关键预测**：这是时滞效应（lagged effect），验证状态是否有持续性

## A5. pro\_celebrity\_chemistry（舞者与选手的"化学反应"指标）

**变量定义：**

python

```
pro_celebrity_chemistry[dancer, celebrity] =
 （该舞者历史上带过该行业选手的次数） / （该舞者总参赛次数）
```

**理论依据：**

- **技能迁移理论**（Baldwin & Ford, 1988）：专业化经验产生类型适配优势
- **领域专长**：带过运动员的舞者 → 知道如何处理高体能但技术粗糙的选手

**对Judge Score的影响：**

- **预期效应**：正向 ( $\beta \approx +0.08 \sim +0.15$ )
- **机制**：行业经验匹配 → 教学针对性强 → 快速弥补选手短板 → 技术进步更快

**对Fan Vote的影响：**

- **预期效应**：弱或无效应 ( $\beta \approx 0 \sim +0.05$ )
- **机制**：粉丝不关心"化学反应"细节，更看重表面效果

**关键预测**：Judge路径单向效应（技术细节的重要性）

## A6. pro\_concurrent\_strength（同赛季其他舞者的平均实力）

**变量定义：**

python

```
pro_concurrent_strength[dancer, season] =
 mean(pro_win_rate for all OTHER dancers in the same season)
```

理论依据：

- 相对评价理论 (Gibbons & Murphy, 1992)：个体表现的评估依赖竞争对手水平
- "大年"/"小年"效应：竞争激烈的赛季 → 中等舞者显得更弱

对Judge Score的影响：

- 预期效应：负向 ( $\beta \approx -0.08 \sim -0.15$ )
- 机制：竞争强度高 → 相对表现下降 → 评委比较中处于劣势

对Fan Vote的影响：

- 预期效应：负向 ( $\beta \approx -0.05 \sim -0.12$ )
- 机制：明星舞者多 → 注意力分散 → 单个舞者的粉丝投票被稀释

关键预测：控制变量，消除赛季竞争强度的混淆效应

---

## B. 名人特征变量 (X\_celeb)

### B1. age\_z (年龄标准化) / age\_group (年龄组别)

变量定义：

python

```
age_z = (celebrity_age - mean_age) / std_age
age_squared = age_z ** 2 # 捕捉非线性U型效应
age_group = pd.cut(age, bins=[20, 30, 40, 50, 60, 100])
```

理论依据：

- 认知老化理论 (Baltes, 1997)：流体智力（学习新技能）20-30岁巅峰，之后下降
- 生命周期粉丝基础：不同年龄段有不同的粉丝群体

对Judge Score的影响：

- 预期效应：负向 ( $\beta \approx -0.02 \sim -0.05$ )，可能有U型（需要平方项）

- 机制：年龄↑ → 体能下降、柔韧性降低、学习速度慢 → 技术动作完成度低 → 评委分数下降

对Fan Vote的影响：

- 预期效应：正向 ( $\beta \approx +0.03 \sim +0.08$ )
- 机制：年长明星 → 怀旧价值 ("童年偶像") + 固定粉丝基础 → 投票忠诚度高

关键预测：

- 效应方向相反 (Judge负, Fan正) → 这是争议的核心来源！
- Jerry Rice、Bristol Palin等案例的理论解释

## B2. industry\_type (行业类别, 原始分类变量)

变量定义：

python

```
industry_type = categorical variable from CSV
Actor/Actress, Singer/Rapper, Athlete, TV Personality, Model, etc.
```

理论依据：

- 职业社会学：不同行业培养不同的身体技能和粉丝基础
- 用于构造其他变量：physicality, visibility等

对Judge Score的影响：

- 预期效应：通过physicality间接作用
- 机制：不同行业 → 不同体能基础 → 不同技术学习速度

对Fan Vote的影响：

- 预期效应：通过visibility/pre\_show\_popularity间接作用
- 机制：不同行业 → 不同媒体曝光 → 不同粉丝规模

关键说明：这是基础分类变量，主要用于构造其他特征

## B3. industry\_physicality (行业体能要求度)

变量定义：

python

```
physicality_map = {
 'Athlete': 3, 'Racing Driver': 3, 'Dancer': 3, # 高体能
 'Actor/Actress': 2, 'Singer/Rapper': 2, 'Model': 2, # 中等
 'TV Personality': 1, 'News Anchor': 1, 'Comedian': 1 # 低体能
}
```

理论依据：

- 技能迁移理论：相关运动经验促进舞蹈学习
- 具身认知理论 (Barsalou, 2008)：身体经验影响动作学习能力
- 运动员优势：肌肉记忆、空间感、节奏感更好

对Judge Score的影响：

- 预期效应：强正向 ( $\beta \approx +0.20 \sim +0.40$ )
- 机制：体能要求高的行业 → 身体控制能力强 → 舞蹈动作学习快、完成度高 → 技术分数显著提升

对Fan Vote的影响：

- 预期效应：弱或无效应 ( $\beta \approx 0 \sim +0.10$ )
- 机制：粉丝不太关心技术细节，更看重明星魅力和表演性

关键预测：

- Judge路径效应 >> Fan路径效应
- 验证"技术vs人气"双路径的分离

## B4. celebrity\_visibility (名人可见度，比较粗糙)

变量定义：

python

```
反向代理：同赛季该行业选手数量的倒数
celebrity_visibility[contestant] = 1 / count(same industry in same season)
逻辑：行业内唯一的明星→可见度高；行业内很多人→可见度被稀释
```

理论依据：

- 注意力竞争理论：媒体关注是零和游戏
- 稀缺性价值：独特性增加吸引力

对Judge Score的影响：

- 预期效应：弱或无效应 ( $\beta \approx 0 \sim +0.05$ )
- 机制：可见度不直接影响技术能力，可能通过自信心间接作用（弱）

对Fan Vote的影响：

- 预期效应：正向 ( $\beta \approx +0.08 \sim +0.15$ )
- 机制：高可见度 → 媒体曝光多 → 粉丝认知度高 → 投票意愿强

关键说明：这是粗糙的代理变量，理想情况需要外部社交媒体数据

## B5. is\_us\_based（是否美国本土）

变量定义：

python

```
is_us_based = 1 if celebrity_hometown == 'United States' else 0
```

理论依据：

- 内群体偏好 (Tajfel & Turner, 1979)：人们倾向支持"自己人"
- 熟悉度效应 (Zajonc, 1968)：重复接触产生积极情感
- 经验证据：Eurovision投票中的邻国互投现象

对Judge Score的影响：

- 预期效应：无效应 ( $\beta \approx 0$ )
- 机制：评委应该保持专业性和客观性，国籍不应影响技术评分

对Fan Vote的影响：

- 预期效应：正向 ( $\beta \approx +0.10 \sim +0.20$ )
- 机制：美国观众 → 对本土明星更熟悉、更有认同感 → 投票倾向性增强

关键预测：

- 只有Fan路径有效应 → 如果显著，证明粉丝投票存在非技术偏好
- 验证投票公平性的关键指标

## B6. pre\_show\_popularity（赛前人气估计）

变量定义（两种方法）：

python

```
方法1: 第一周超额表现
pre_show_popularity = fan_vote_rank_week1 - judge_rank_week1

方法2: 行业历史平均
pre_show_popularity = mean(fan_votes for same industry in past seasons)
```

#### 理论依据:

- 前期声誉理论 (Groysberg et al., 2004): 个体携带"声誉资本"进入新环境
- 第一周逻辑: 评委还没偏见 (客观) vs 粉丝已有先验 (主观) → 差异=赛前人气

#### 对Judge Score的影响:

- 预期效应: 无效应或弱正向 ( $\beta \approx 0 \sim +0.05$ )
- 机制: 赛前人气不直接影响技术, 可能通过自信心略微提升 (心理效应)

#### 对Fan Vote的影响:

- 预期效应: 强正向 ( $\beta \approx +0.15 \sim +0.30$ )
- 机制: 赛前人气高 → 固定粉丝基础大 → 投票"地板"高 → 持续整个赛季

#### 关键预测:

- Fan单向强效应
- 控制这个变量后, 可以分离"初始人气"和"比赛中赢得的人气"

## B7. underdog\_index (黑马指数)

#### 变量定义:

python

```
underdog_index = actual_placement - expected_placement
```

其中 expected\_placement 基于:

- 第一周judge分数预测的名次
- 或基于行业/年龄的回归预测

#### 理论依据:

- 惊喜效应 (Meyer et al., 1997): 超出预期的表现产生更强的情感反应
- 叙事偏好: 观众喜欢"逆袭"故事

#### 对Judge Score的影响:



- 预期效应：这是结果变量，不是原因
- 逻辑：黑马是因为judge分数提升幅度大，不是黑马导致分数高

对Fan Vote的影响：

- 预期效应：正向 ( $\beta \approx +0.10 \sim +0.20$ )
- 机制：超预期表现 → 媒体报道多 ("黑马"标签) → 吸引新粉丝 → 投票增加

关键说明：

- 可作为中介变量：好表现→黑马效应→更多粉丝
- 或作为控制变量：控制"惊喜效应"后看其他因素的影响

## B8. controversy\_potential（争议潜力）

变量定义（基于第二问的safety margin）：

python

```
方法1：基于第二问结果
controversy_potential = -safety_margin # margin越小，争议越大

方法2：judge-fan分歧
controversy_potential = abs(judge_rank - fan_rank) / n_contestants

方法3：累积争议
controversy_potential = sum(abs(judge_rank[t] - fan_rank[t]) for t in 1..T)
```

理论依据：

- 认知失调理论 (Festinger, 1957)：judge评价和fan评价冲突 → 心理不适 → 引发讨论/争议
- 争议与关注度：争议选手获得更多媒体曝光

对Judge Score的影响：

- 预期效应：可能负向 ( $\beta \approx -0.05 \sim -0.15$ )
- 机制：争议选手通常是因为技术不足但人气高 → judge分数相对较低

对Fan Vote的影响：

- 预期效应：可能正向 ( $\beta \approx +0.10 \sim +0.25$ )
- 机制：争议 → 媒体曝光↑ → 粉丝极化（铁粉更忠诚） → 投票可能增加

关键用途：

- 作为控制变量：控制争议后，检验其他因素的"纯效应"
- 作为结果变量：分析哪些因素（年龄、行业等）导致争议
- 整合第二问：safety margin最小的周 = 争议最高的周